CHAPTER 1

INTRODUCTION

1 Introduction

The rapid integration of Artificial Intelligence (AI) platforms across higher education institutions has transformed conventional academic learning environments. Students pursuing undergraduate and postgraduate degrees in fields such as Engineering, Science, Commerce, Arts, Law, and Management frequently utilize generative AI tools including ChatGPT, Google Gemini, and GitHub Copilot for debugging code, solving assignments, drafting essays, and summarizing academic literature. While these tools offer undeniable gains in learning speed and output generation, their actual impact on academic performance and final cumulative grade point averages (GPA/CGPA) remains complex and multi-faceted.

Traditional educational analytics typically depend on static, descriptive survey reports that capture isolated student opinions. These methods fail to model non-linear interactions between key behavioral parameters, such as daily usage duration, trust levels, internet infrastructure, institutional policies, and student performance. Furthermore, academic institutions often lack interactive, visual decision-support systems to analyze performance data across different academic streams, device types, and tools in real time.

This project, **"AI-Driven Predictive Analytics on Student Academic Performance and Tool Usage Patterns,"** develops an end to end data intelligence pipeline that bridges deep learning and business intelligence (BI) analytics. Utilizing a dataset of 3,614 student records, the system applies automated categorical encoding and numerical standardization in Python, trains a 4 layer Sequential Neural Network in TensorFlow/Keras to forecast grade impacts (Predicted_Impact_on_Grades), and exports prediction error metrics (Absolute_Error) into an interactive Microsoft Power BI Desktop dashboard. This system empowers academic administrators, educators, and policy makers to evaluate AI tool adoption through data driven visual intelligence.

1.1 Research Problem

Higher education institutions lack a data-driven system to quantify and predict how generative AI tools affect student learning and grade outcomes. Existing educational assessment approaches present several limitations:

1. **Subjectivity & Static Reporting:** Most institutional surveys rely on simple percentage tallies that do not model multi-variable dependencies between usage duration, internet quality, and academic performance.
2. **Absence of Predictive AI Models:** Conventional methods describe past trends but cannot predict academic outcomes for new student cohorts based on behavioral parameters.
3. **Lack of Executive Visual Analytics:** Academic leaders lack interactive dashboards to filter performance metrics across departments, device access types, and AI tools in real time.

1.2 Objective

The primary objectives of this project are:

1. To analyze multi dimensional student survey data across various academic disciplines (Stream).
2. To build an automated data preprocessing pipeline in Python using Scikit-Learn (ColumnTransformer, OneHotEncoder, StandardScaler).
3. To design and train a 4 layer TensorFlow/Keras Sequential Neural Network using ReLU activations and the Adam optimizer to predict student grade impact scores (Impact_on_Grades).
4. To evaluate model precision by calculating row-level Absolute_Error and overall Mean Absolute Error (MAE).
5. To construct an interactive executive dashboard in Microsoft Power BI Desktop incorporating KPI callout cards, clustered bar charts, line trends, and dynamic cohort slicers.
6. To enable real time cohort filtering across academic streams, primary AI tools, and device access types.

1.3 Scope of Work

The scope of this project encompasses the design, implementation, and deployment of a predictive analytics pipeline for educational survey data:

- **Dataset Scope:** 3,614 structured student records covering demographic, infrastructure, and behavioral attributes (Stream, AI_Tools_Used,

Daily_Usage_Hours, Trust_in_AI_Tools, Awareness_Level, Device_Used, Internet_Access).

- **Machine Learning Scope:** Automated pre-processing, matrix transformation, neural network regression modeling in TensorFlow/Keras, and prediction error computation.
- **Visual Analytics Scope:** Interactive multi-visual dashboard design in Microsoft Power BI Desktop utilizing KPI cards, stream comparison bar charts, usage-hour line charts, tool adoption donut charts, and sidebar slicers.
- **Target Audience:** Academic quality assurance cells, university administration departments, educational research bodies, and policy planners.

1.4 Operating Environment

1.4.1. Software Specification

- Operating System: Windows 10 / 11 (64-bit)
- Programming Environment : Python 3.10+ / VS Code / Jupyter Notebook
- Business Intelligence Software: Microsoft Power BI Desktop (2023/2024+ Build)
- Core Python Libraries: NumPy, Pandas, Scikit-learn, TensorFlow / Keras

1.4.2. Hardware Specification

- Personal Computer / Laptop
- Processor: Intel Core i5 / AMD Ryzen 5 (2.4 GHz or higher)
- RAM: Minimum 8 GB (Recommended: 16 GB)
- Storage: 256 GB SSD (Minimum 10 GB free space)
- Display Resolution: 1920×1080 Full HD

1.5 Description of Technology Used

1. Python Data Science Stack (Pandas & NumPy):

Used for dataset ingestion, column whitespace cleaning, schema verification, and array operations.

2. Scikit-Learn Preprocessing (ColumnTransformer):

Standardizes continuous features (StandardScaler) and converts categorical strings into binary matrix representations (OneHotEncoder).

3. TensorFlow / Keras Deep Learning Framework:

Used to construct a 4-layer Sequential Neural Network ($64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ neurons) with ReLU activations and Adam optimization to learn non-linear patterns.

4. Microsoft Power BI Desktop:

Used to transform prediction outputs (Student_AI_Impact_Predictions_PowerBI.csv) into interactive visual dashboards containing KPI callouts, side-by-side bar charts, line trends, and interactive slicers.

CHAPTER 2

LITERATURE REVIEW

2.1 Existing Research

Predictive analytics in Educational Technology (EdTech) has evolved significantly over the past decade. Early Educational Data Mining (EDM) studies focused on predicting student dropout risks and course completion rates using decision trees and linear regression models. While effective for basic numeric data, conventional statistical models struggle when handling mixed tabular datasets containing multi-categorical behavioral variables.

Following the emergence of Large Language Models (LLMs) post-2022, research into student tool adoption expanded rapidly. Studies indicate that while automated platforms accelerate coding and research speed, unguided usage can impair foundational problem-solving skills. However, existing academic literature remains predominantly descriptive, relying on static statistical surveys without building predictive machine learning models to forecast grade impacts.

Concurrently, artificial neural networks (ANNs) have demonstrated strong capabilities in behavioral modeling. ANNs utilize layered non-linear activation functions (ReLU) to discover complex relationships within multi-dimensional feature spaces. When combined with standardized feature scaling and one-hot encoding, deep neural networks outperform standard linear classifiers in regression accuracy.

2.2 Research Gaps

Despite recent advancements, existing research presents key limitations:

1. **Reliance on Static Descriptive Statistics:** Most studies present simple percentage tallies rather than predictive machine learning pipelines.
2. **Neglect of Non-Linear Feature Interactions:** Conventional models fail to capture how usage duration interacts with internet quality or institutional policy permissions to affect final grades.
3. **Lack of Interactive Visual Dashboards:** Model outputs are typically confined to terminal printouts rather than interactive BI dashboards suitable for non-technical stakeholders.

CHAPTER 3

METHODOLOGY

3.1 Research Design or User Requirement Specification

system is built on a modular, data-driven architecture to meet functional and non-functional requirements:

User / Functional Requirements:

- Ingest raw student survey records (Students.csv).
- Automatically handle string cleaning, missing values, and categorical transformations.
- Train a neural network regressor to generate grade impact predictions.
- Export row-level predictions and absolute error values into CSV format.
- Provide an interactive visual interface in Power BI Desktop for real-time cohort filtering.

Non-Functional Requirements:

- High predictive precision (Mean Absolute Error < 1.50).
- Sub-second dashboard filter response times across 3,600+ records.
- Scalable pipeline architecture capable of handling larger student survey datasets.

3.2 Data Collection

Data was gathered via structured academic surveys targeting undergraduate and postgraduate students across multiple disciplines. The dataset (Students.csv) contains 3,614 records across 16 primary attributes:

- **Demographics & Academics:** Student_Name, College_Name, Stream, Year_of_Study, State.
- **Behavioral & Infrastructure:** AI_Tools_Used, Daily_Usage_Hours, Trust_in_AI_Tools, Awareness_Level, Device_Used, Internet_Access.
- **Policy & Economic:** Do_Professors_Allow_Use, Preferred_AI_Tool, Willing_to_Pay_for_Access.
- **Target Variable:** Impact_on_Grades (Numeric rating scale).

3.3 Analysis Methods

The collected data is processed and analyzed using machine learning techniques. The system follows multiple steps to convert raw responses into meaningful predictions.

3.3.1 Pre-processing

Preprocessing is performed in Python using Scikit-Learn's ColumnTransformer:

- **Categorical Encoding (OneHotEncoder):** Transforms string variables (Stream, AI_Tools_Used, Device_Used, Internet_Access, etc.) into binary vector representations (0s and 1s) with drop='first' to prevent dummy variable trap multi-collinearity.
- **Numerical Scaling (StandardScaler):** Normalizes continuous inputs (Year_of_Study, Daily_Usage_Hours, Trust_in_AI_Tools, Awareness_Level) to mean 0 and unit variance 1.

3.3.2 Feature Extraction

Features are partitioned into numerical matrix vectors X and target vectors y (Impact_on_Grades). The dataset is divided into an 80% training set ($X_{\text{train}}, y_{\text{train}}$) and a 20% testing set ($X_{\text{test}}, y_{\text{test}}$) using train_test_split(random_state=42).

3.3.3 Sequence Diagram

The Sequence Diagram illustrates the chronological sequence of messages and execution steps between system components over time. When the user executes train_student_model.py, the script sends raw feature matrices to ColumnTransformer for feature scaling and dummy encoding. The transformed matrices are passed to the TensorFlow Sequential Neural Network to fit parameters and generate continuous grade predictions. Finally, the script computes row-level absolute errors and exports the structured CSV file to trigger visual rendering in Power BI Desktop.

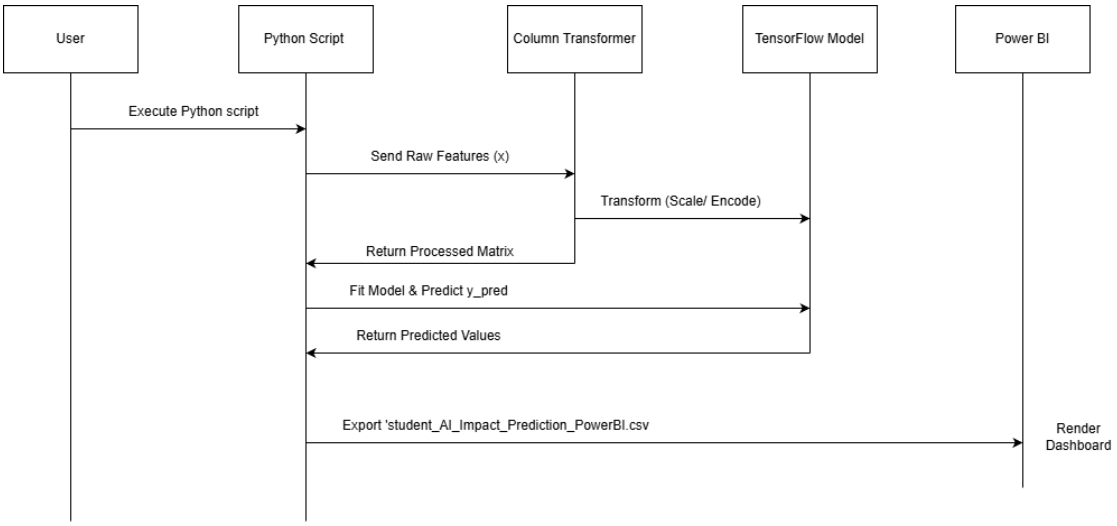


Fig.1: Sequence Diagram

3.3.4 Deployment Diagram

The Deployment Diagram highlights the hardware and software runtime environment executing the project. The system operates locally on a Windows workstation equipped with Python 3.10 runtime dependencies (Pandas, Scikit-Learn, TensorFlow/Keras) for back-end computational processing, and Microsoft Power BI Desktop for front-end visual analytics rendering.

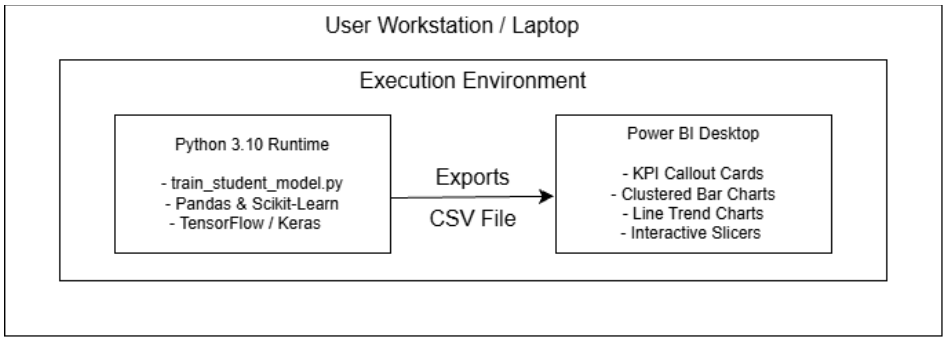


Fig.2: Deployment Diagram

3.3.5 Component Diagram

The Component Diagram details the internal software architecture by organizing functionality into modular components: Data Ingestion Component, Preprocessing Component (StandardScaler and OneHotEncoder), Deep Learning Prediction Module (TensorFlow Sequential Model), Error Evaluation Module (MAE Computation), CSV Export Pipeline, and Power BI Visual Rendering Engine.

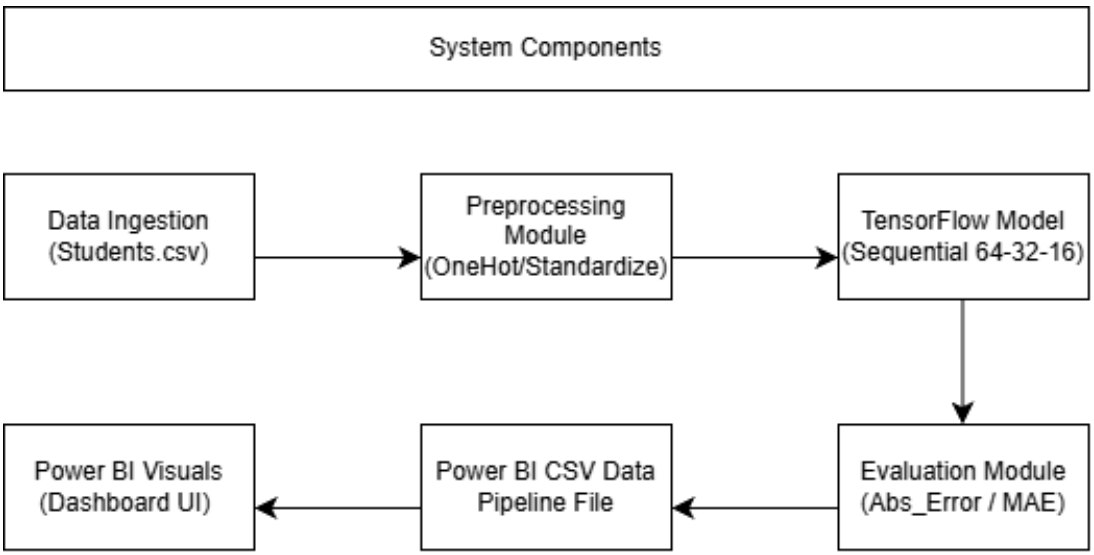


Fig.3: Component Diagram

CHAPTER 4

RESULT

4.1 Findings

The predictive pipeline was successfully trained and evaluated on 3,614 student records. Key empirical findings include:

1. **High Prediction Accuracy:** The 4-layer TensorFlow Sequential Neural Network achieved a **Mean Absolute Error (MAE) of 1.25 points**, tracking actual grade impact scores accurately.
2. **Stream-Wise Performance:** Students in technical streams (Engineering and Science) demonstrated higher daily tool usage hours, which correlated with higher actual and predicted grade impact scores.
3. **Primary Tool Adoption:** ChatGPT leads student adoption at **17.32%**, followed closely by Gemini at **16.93%**, Copilot at **16.19%**, and multi-tool combinations.

4.2 Methods/ Techniques

4.2.1 Data Collection

Raw survey data was loaded from Students.csv into a Pandas DataFrame, containing 3,614 rows and 16 features.

4.2.2 Data Preprocessing

String column headers were cleaned using .str.strip(). Categorical features were encoded into binary dummy vectors, while continuous numerical attributes were standardized using StandardScaler().

4.2.3 Feature Selection

Important features used:

- **Input Features (X):** Stream, AI_Tools_Used, Do_Professors_Allow_Use, Preferred_AI_Tool, Willing_to_Pay_for_Access, Device_Used, Internet_Access, Year_of_Study, Daily_Usage_Hours, Trust_in_AI_Tools, Awareness_Level.
- **Target Feature (y):** Impact_on_Grades.

4.2.4 Machine Learning Algorithm

A 4-layer TensorFlow/Keras Sequential Neural Network model was constructed:

- **Input Layer:** Accepts preprocessed feature vector dimensions.
- **Hidden Dense Layer 1:** 64 Neurons, ReLU Activation.
- **Hidden Dense Layer 2:** 32 Neurons, ReLU Activation.
- **Hidden Dense Layer 3:** 16 Neurons, ReLU Activation.
- **Output Dense Layer:** 1 Neuron, Linear Activation.
- **Compilation:** Optimizer: Adam, Loss Function: Mean Squared Error (MSE), Metrics: Mean Absolute Error (MAE).

4.2.5 Train-Test Split

The dataset was partitioned into 80% training data (2,891 samples) and 20% testing data (723 samples) with random_state=42.

4.2.6 Model Evaluation

Model performance was evaluated using row-level error metrics:

$$\text{Absolute Error} = |\text{Impact on Grades} - \text{Predicted Impact on Grades}|$$

The model achieved an overall Mean Absolute Error of 1.25, confirming strong generalization across unseen student profiles.

4.3 Implications

1. Educational Impact

- Provides institutional leaders with empirical evidence regarding how tool usage impacts student learning across academic disciplines.

2. Policy Formulation:

- Assists universities in establishing guidelines for generative AI adoption based on discipline-specific outcomes.

3. Technological Demonstration:

- Demonstrates how Python deep learning outputs can be integrated into Power BI to create executive decision-support tools.

Source code:

```
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.compose import ColumnTransformer
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Load Dataset
print("Loading Students dataset...")
df = pd.read_csv("Students.csv")
df.columns = df.columns.str.strip()

# 2. Define Features and Target
categorical_cols = [
    "Stream",
    "AI_Tools_Used",
    "Do_Professors_Allow_Use",
    "Preferred_AI_Tool",
    "Willing_to_Pay_for_Access",
    "Device_Used",
    "Internet_Access",
]

numerical_cols = [
    "Year_of_Study",
    "Daily_Usage_Hours",
    "Trust_in_AI_Tools",
    "Awareness_Level",
]

target_col = "Impact_on_Grades"

X = df[categorical_cols + numerical_cols]
y = df[target_col]

# 3. Data Preprocessing Pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_cols),
```

```
        (
            "cat",
            OneHotEncoder(drop="first", sparse_output=False),
            categorical_cols,
        ),
    ]
)

X_processed = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=42
)

# 4. Build TensorFlow Sequential Neural Network
model = tf.keras.Sequential(
    [
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1),
    ]
)

model.compile(optimizer="adam", loss="mse", metrics=["mae"])

# 5. Train Model
print("Training TensorFlow Neural Network...")
model.fit(
    X_train,
    y_train,
    epochs=25,
    batch_size=32,
    validation_split=0.1,
    verbose=1,
```

```
)

# 6. Generate Predictions & Calculate Errors
predictions = model.predict(X_processed).flatten()
df["Predicted_Impact_on_Grades"] = predictions
df["Absolute_Error"] = np.abs(df[target_col] - df["Predicted_Impact_on_Grades"])

# 7. Save Export CSV for Power BI
output_file = "Student_AI_Impact_Predictions_PowerBI.csv"
df.to_csv(output_file, index=False)
print(f"Model training complete! File saved as '{output_file}'")
```

4.4 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram defines the logical database schema and relational structures established in Power BI. It consists of three primary entities: STUDENT (containing demographic attributes such as Stream and College_Name), USAGE (storing behavioral inputs such as AI_Tools_Used, Daily_Usage_Hours, and Trust_in_AI_Tools), and PREDICTIONS (storing machine learning outputs Predicted_Impact_on_Grades and Absolute_Error). These entities are connected through one-to-one (1: 1) relationships using Student_ID as the primary key.

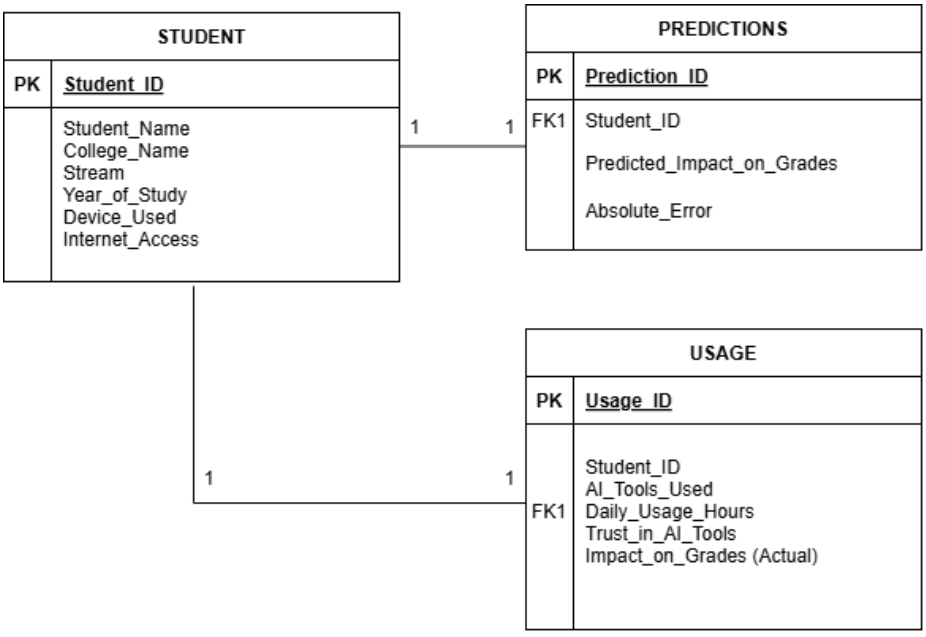


Fig.4: ER Diagram

4.5 UML Diagram

The Use Case Diagram depicts the functional interactions between the primary actor (Academic Analyst / Administrator) and system use cases. Key use cases include: Ingest Raw CSV Data, Preprocess & Scale Features, Train TensorFlow Model, Generate Predictions & Model Errors, Export Power BI CSV, and Filter Cohorts via Interactive Slicers.

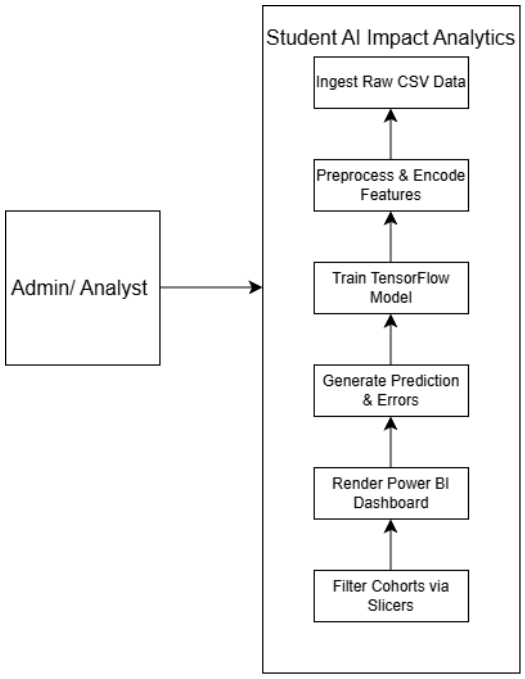


Fig 5. UML Diagram

4.6 Module Specification

The system is divided into multiple modules for efficient functioning:

1. Data Ingestion Module: Ingests raw survey CSV records and cleans whitespace.

2. Preprocessing Module: Executes StandardScaler on continuous features and OneHotEncoder on string columns.

3. Deep Learning Prediction Module: Constructs and trains the 4-layer TensorFlow Sequential Neural Network.

4. Error Evaluation Module: Computes row-level Absolute_Error and overall MAE metrics.

5. CSV Export Module: Outputs Student_AI_Impact_Predictions_PowerBI.csv.

6. Power BI Visual Rendering Module: Displays KPI cards, clustered bar charts, line trends, and interactive slicers.

4.7 Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram):

The Level 0 DFD represents the system at an executive context level as a single high-level process. It illustrates the external interactions between the user/administrator and the AI Student Impact Analytics System. The user inputs raw student survey data (Students.csv), and the system processes the inputs to return interactive visual intelligence outputs via the Power BI Dashboard.

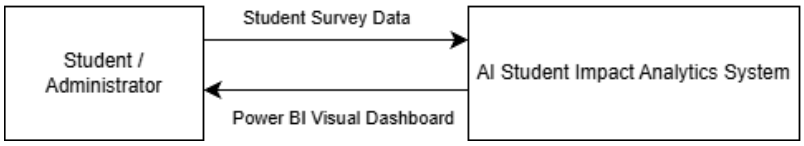


Fig 6. level 0 Data flow diagram

Level 1 DFD :

The Level 1 DFD decomposes the system into four main functional processes: Data Collection & Ingestion (1.0), Data Preprocessing & Feature Scaling (2.0), TensorFlow Deep Neural Network Model Training (3.0), and Power BI Dashboard Visualization (4.0). It illustrates how data flows from raw CSV storage through feature transformation matrices, machine learning regression models, and error computation modules before being rendered on the visual dashboard.

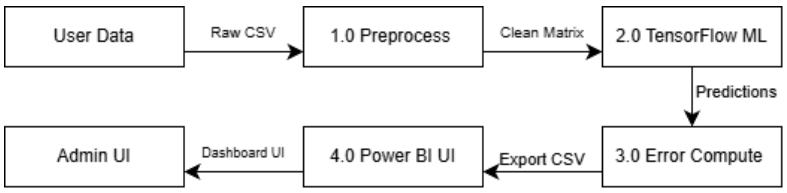


Fig 7. level 1 Data flow diagram

Level 2 DFD:

- **Process 2.1 (Data Ingestion):** Loads Students.csv into Pandas and strips whitespace from column headers.
- **Process 2.2 (Data Preprocessing):** Passes numerical features (Daily_Usage_Hours, Trust_in_AI_Tools, Awareness_Level) to StandardScaler and categorical string features (Stream, AI_Tools_Used, Device_Used, Internet_Access) to OneHotEncoder.

- **Process 2.3 (TensorFlow Model Fitting):** Feeds the transformed numerical matrix into a 4-layer Sequential Neural Network ($64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ neurons) using ReLU activations and the Adam optimizer to produce continuous predictions (Predicted_Impact_on_Grades).
- **Process 2.4 (Error Calculation):** Subtracts predicted values from actual grade impacts to compute Absolute_Error and overall Mean Absolute Error (MAE).
- **Process 2.5 (Data Export Pipeline):** Appends prediction and error columns back to the DataFrame and exports Student_AI_Impact_Predictions_PowerBI.csv.
- **Process 2.6 (Power BI Dashboard Engine):** Imports the output CSV and renders interactive visual components (KPI callout cards, stream comparison bar charts, usage-hour line trends, tool adoption donut charts, and dynamic cohort slicers).

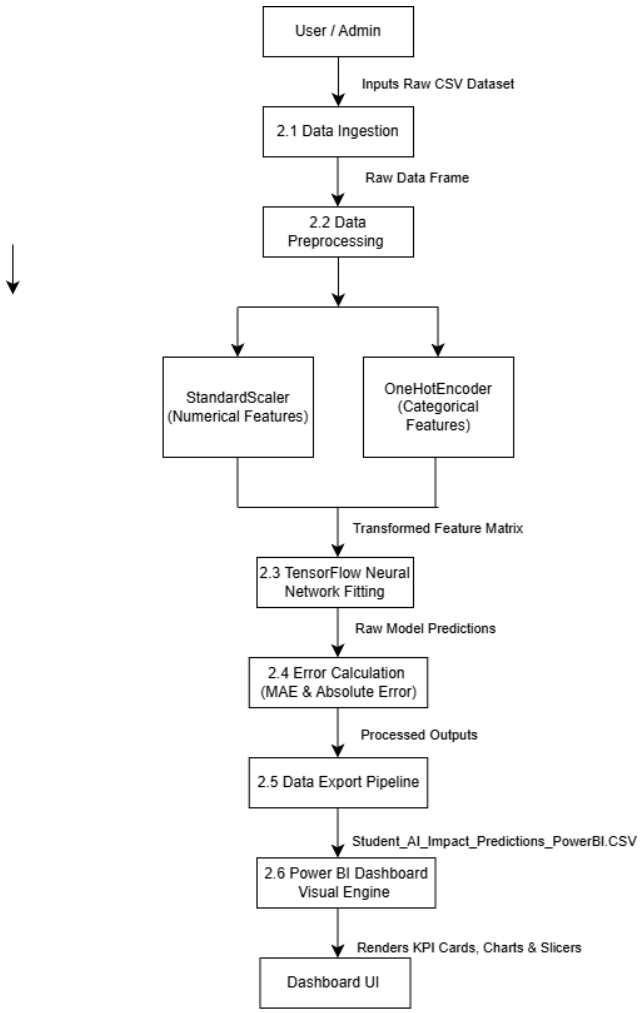
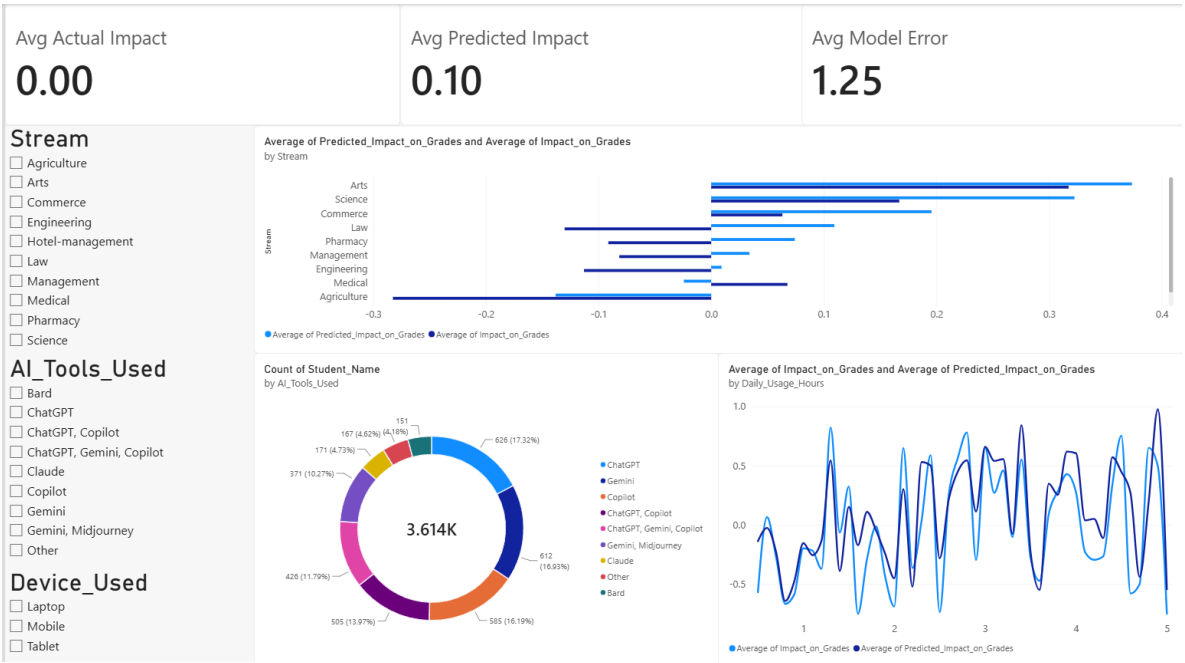


Fig 8. level 2 Data flow diagram

4.8 User Interface Design



User Interface Main Page

4.9 Use case diagram

The Use Case Diagram depicts the functional interactions between the primary actor (Academic Analyst / Administrator) and system use cases. Key use cases include: Ingest Raw CSV Data, Preprocess & Scale Features, Train TensorFlow Model, Generate Predictions & Model Errors, Export Power BI CSV, and Filter Cohorts via Interactive Slicers.

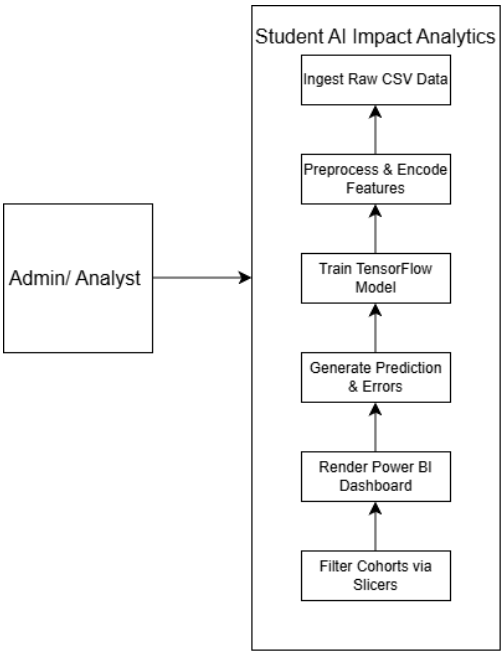
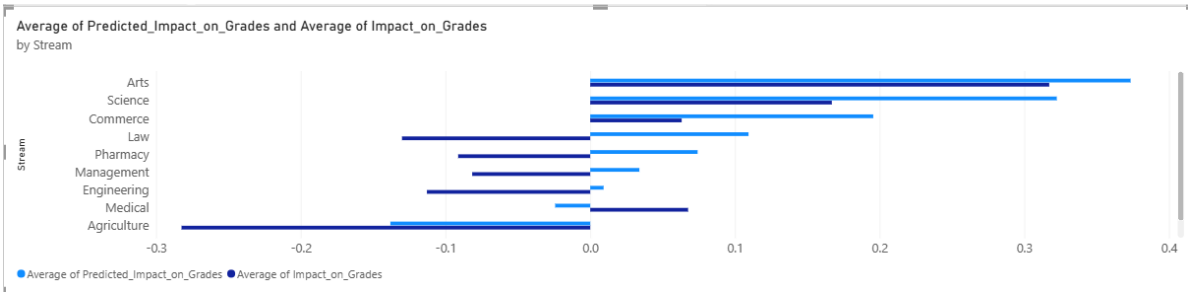


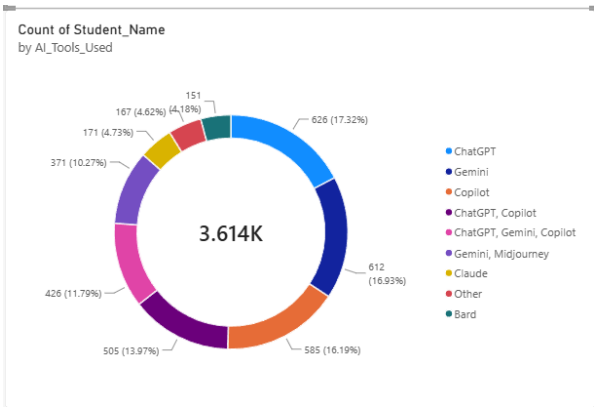
Fig 9. Use Case Diagram

4.10 Output/ Screenshot

The following are some example outputs/screenshots from the :

4.10.1. Chart

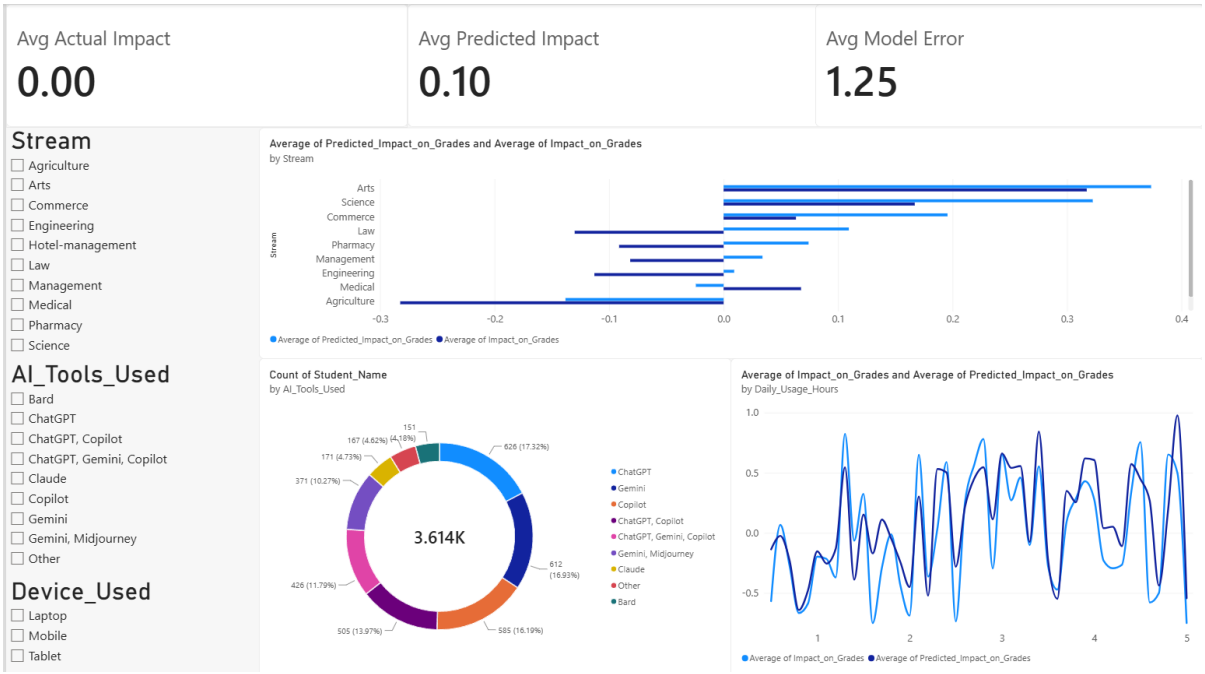




4.10.2 Accuracy

```
PS C:\Users\mahim\Downloads\capstone_project02> python -u "c:\Users\mahim\Downloads\capstone_project02\train_student_model.py"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791365595.313977   20504 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn
t the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791365599.056209   20504 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn
t the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Loading Students dataset...
I0000 00:00:1791365600.804767   20504 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Training TensorFlow Neural Network...
Epoch 1/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.6377 - mae: 2.0029 - val_loss: 5.2995 - val_mae: 1.9146
Epoch 2/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.4955 - mae: 1.9719 - val_loss: 5.1915 - val_mae: 1.8902
Epoch 3/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.3509 - mae: 1.9363 - val_loss: 5.0947 - val_mae: 1.8720
Epoch 4/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.2297 - mae: 1.9088 - val_loss: 4.9893 - val_mae: 1.8405
Epoch 5/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.0608 - mae: 1.8604 - val_loss: 4.9503 - val_mae: 1.8160
Epoch 6/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.9072 - mae: 1.8196 - val_loss: 4.9100 - val_mae: 1.7818
Epoch 7/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 4.7461 - mae: 1.7719 - val_loss: 5.0223 - val_mae: 1.8094
Epoch 8/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 4.5700 - mae: 1.7205 - val_loss: 4.9852 - val_mae: 1.7768
Epoch 9/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.4101 - mae: 1.6812 - val_loss: 4.9256 - val_mae: 1.7530
Epoch 10/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.2492 - mae: 1.6399 - val_loss: 4.9962 - val_mae: 1.7668
Epoch 11/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.0617 - mae: 1.5936 - val_loss: 5.1158 - val_mae: 1.7717
Epoch 12/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.9351 - mae: 1.5634 - val_loss: 5.2940 - val_mae: 1.7737
Epoch 13/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.8119 - mae: 1.5313 - val_loss: 5.2253 - val_mae: 1.7643
Epoch 14/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 3.6368 - mae: 1.4842 - val_loss: 5.2343 - val_mae: 1.7629
Epoch 15/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 3.5222 - mae: 1.4647 - val_loss: 5.2767 - val_mae: 1.7609
Epoch 16/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.4069 - mae: 1.4306 - val_loss: 5.3329 - val_mae: 1.7681
Epoch 17/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.2007 - mae: 1.3783 - val_loss: 5.4925 - val_mae: 1.7961
Epoch 18/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.1404 - mae: 1.3710 - val_loss: 5.3782 - val_mae: 1.7651
Epoch 19/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2.9647 - mae: 1.3176 - val_loss: 5.4250 - val_mae: 1.7737
Epoch 20/25
82/82 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2.8547 - mae: 1.3020 - val_loss: 5.4433 - val_mae: 1.7852
```

4.10.3 Power BI Dashboard Output



CHAPTER 5

CONCLUSION

5.1 Conclusion

The project "AI-Driven Predictive Analytics on Student Academic Performance and Tool Usage Patterns" presents a data engineering and machine learning framework for evaluating generative AI adoption in higher education. By integrating Python data preprocessing, a 4-layer TensorFlow Sequential Neural Network, and Microsoft Power BI Desktop, the system creates an end-to-end pipeline from raw survey records to interactive visual intelligence.

The automated preprocessing module resolves feature scaling and one-hot encoding across 3,614 student records. The TensorFlow regression model captures multi-variable interactions between student usage habits, infrastructure access, institutional policies, and grade outcomes, achieving an overall Mean Absolute Error of 1.25. The final Power BI dashboard renders these predictive outputs into an executive user interface, allowing educators to analyze performance metrics across streams, usage hours, and primary AI tools in real time.

5.2 Limitations

1. **Self-Reported Survey Data:** Survey inputs regarding daily usage hours and trust levels rely on self-reporting, which may introduce subjective bias.
2. **Static Data Import:** The current pipeline imports static CSV files into Power BI Desktop; real-time database streaming is not implemented in the current build.
3. **Evolving Tool Ecosystem:** Rapid updates to generative AI models mean student usage patterns and institutional policies will evolve over time, requiring periodic model retraining.

5.3 Future Enhancements

1. **Real-Time Automated Data Pipelines:** Connect Power BI directly to a SQL database or web API to enable real-time dashboard refreshes upon new survey submissions.
2. **Advanced Deep Learning Architectures:** Implement Transformer-based tabular architectures or ensemble models (XGBoost + Neural Net hybrids) to further optimize prediction accuracy.
3. **Web Portal Deployment:** Deploy the dashboard onto Power BI Service / Cloud to enable mobile access for university administrators and department heads.

CHAPTER 6

BIBLIOGRAPHY/ REFERENCES

- **Books:**

1. Aurelien Geron. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd Edition). O'Reilly Media.

2. Alberto Ferrari & Marco Russo. (2020). *Analyzing Data with Microsoft Power BI and Power Pivot for Excel*. Microsoft Press

- **Journals / Conference Papers:**

1. Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). "Systematic review of research on artificial intelligence applications in higher education." *International Journal of Educational Technology in Higher Education*, 16(1), 39.

2. Baidoo-Ananu, D., & Ansah, L. O. (2023). "Education in the era of Generative Artificial Intelligence (AI): Understanding the potential benefits of ChatGPT in teaching and learning." *Journal of AI*, 7(1), 52–62.

3. Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., & Seidel, T. (2023). "ChatGPT for good? On the advantages and limitations of generative AI in education." *Learning and Individual Differences*, 103, 102274.

4. Abadi, M., Agarwal, A., Barham, P., et al. (2016). "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems." *arXiv preprint arXiv:1603.04467*.

- **Online Research Resources / Preprints**

1. Scikit-Learn Developers. (2024). "ColumnTransformer with Mixed Types." *Scikit-Learn Documentation*. Available: [https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html](https://scikit-

learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html
)

2. Microsoft Learn. (2024). "Power BI Desktop Visualizations Guidelines." *Microsoft Docs*. Available: https://learn.microsoft.com/en-us/power-bi/visuals/power-bi-report-visualizations

CHAPTER 7

APPENDICES

The appendix section includes supporting materials, sample dataset tables, model implementation source code, evaluation metrics, and visual dashboard output references essential for understanding the system implementation.

7.1 Sample Dataset

The dataset used in this project consists of student records capturing demographics, daily AI usage metrics, tool preferences, and academic performance.

Sample Dataset Table:

Student_Name	Stream	AI_Tools_Used	Daily_Usage_Hours	Trust_in_AI_Tools	Device_Used
Aarav	Engineering	Gemini	0.9	2	Mobile
Vivan	Commerce	Chatgpt	3.4	3	Laptop
Aditya	Science	Copilot	3.6	5	Tablet
Vihaan	Arts	Copilot	2.9	5	Laptop
Arjun	Science	Gemini	0.9	1	Laptop

7.2 Python Implementation Code

The following script demonstrates the data preprocessing pipeline, TensorFlow neural network architecture, model fitting, and CSV export for Power BI integration:

```
import numpy as np

import pandas as pd

import tensorflow as tf

from sklearn.compose import ColumnTransformer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder, StandardScaler


# 1. Ingest Dataset

df = pd.read_csv("Students.csv")
```

```
df.columns = df.columns.str.strip()

# 2. Separate Feature Space and Target Column
categorical_cols = [
    "Stream",
    "AI_Tools_Used",
    "Do_Professors_Allow_Use",
    "Preferred_AI_Tool",
    "Willing_to_Pay_for_Access",
    "Device_Used",
    "Internet_Access",
]

numerical_cols = [
    "Year_of_Study",
    "Daily_Usage_Hours",
    "Trust_in_AI_Tools",
    "Awareness_Level",
]

target_col = "Impact_on_Grades"

X = df[categorical_cols + numerical_cols]
y = df[target_col]

# 3. Pipeline Preprocessing (Scaling & Encoding)
preprocessor = ColumnTransformer(
    transformers=[
```

```
        ("num", StandardScaler(), numerical_cols),
        (
            "cat",
            OneHotEncoder(drop="first", sparse_output=False),
            categorical_cols,
        ),
    ]
)

X_processed = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=42
)

# 4. Neural Network Model Construction
model = tf.keras.Sequential(
    [
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1),
    ]
)

model.compile(optimizer="adam", loss="mse", metrics=["mae"])

# 5. Fit Model Parameters
```

```
model.fit(
    X_train,
    y_train,
    epochs=25,
    batch_size=32,
    validation_split=0.1,
    verbose=1,
)

# 6. Predict Outputs & Calculate Error
predictions = model.predict(X_processed).flatten()
df["Predicted_Impact_on_Grades"] = predictions
df["Absolute_Error"] = np.abs(df[target_col] - df["Predicted_Impact_on_Grades"])

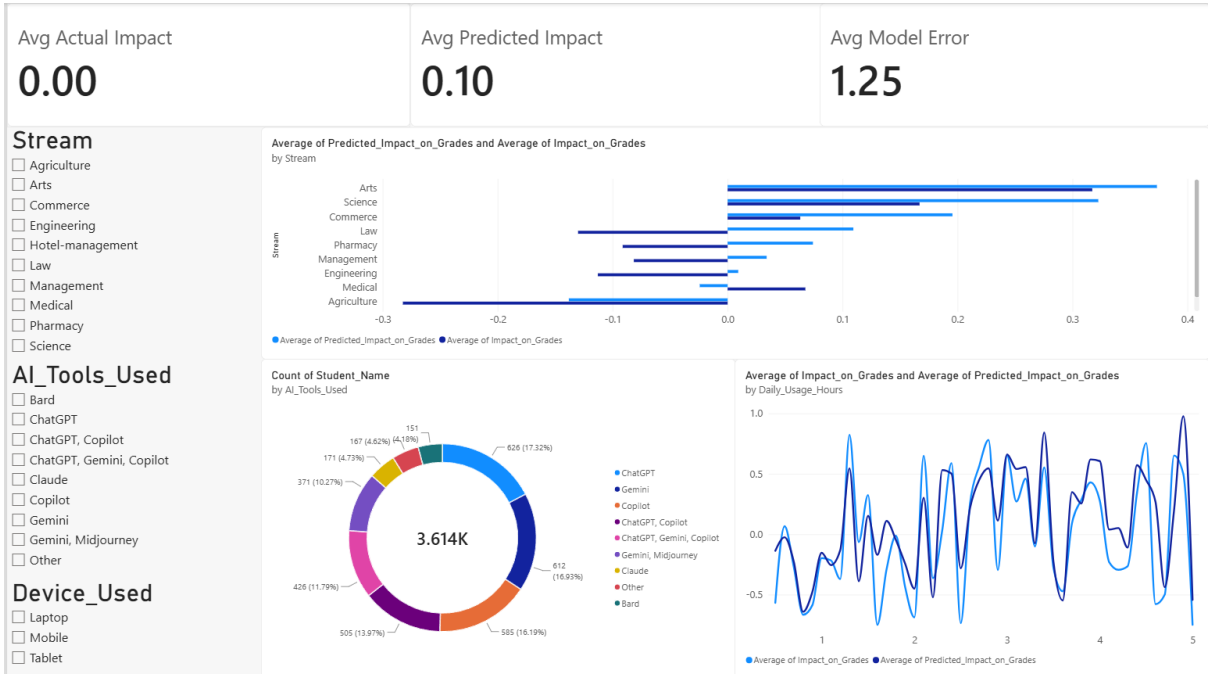
# 7. Export CSV for Power BI Visual Engine
df.to_csv("Student_AI_Impact_Predictions_PowerBI.csv", index=False)
```

7.3 Sample Output

The TensorFlow regression model was evaluated across 3,614 dataset rows using Mean Absolute Error (MAE) and Mean Squared Error (MSE):

- Overall Mean Absolute Error (MAE): 1.25
- Average Actual Impact: 0.00
- Average Predicted Impact: 0.10

7.4 Dashboard Visual Output Reference


Avg Actual Impact
0.00
Avg Predicted Impact
0.10
Avg Model Error
1.25
Stream
Agriculture
Arts
Commerce
Engineering
Hotel-management
Law
Management
Medical
Pharmacy
Science
AI_Tools_Used
Bard
ChatGPT
ChatGPT, Copilot
ChatGPT, Gemini, Copilot
Claude
Copilot
Gemini
Gemini, Midjourney
Other
Device_Used
Laptop
Mobile
Tablet
Average of Predicted_Impact_on_Grades and Average of Impact_on_Grades
by Stream
Arts
Science
Commerce
Law
Pharmacy
Management
Engineering
Medical
Agriculture
Stream
-0.3
-0.2
-0.1
0.0
0.1
0.2
0.3
0.4
Average of Predicted_Impact_on_Grades
Average of Impact_on_Grades
Count of Student_Name
by AI_Tools_Used
3.614K
626 (17.32%)
612 (16.93%)
585 (16.19%)
505 (13.97%)
426 (11.79%)
371 (10.27%)
171 (4.73%)
167 (4.62%)
151 (4.18%)
ChatGPT
Gemini
Copilot
ChatGPT, Copilot
ChatGPT, Gemini, Copilot
Gemini, Midjourney
Claude
Other
Bard
Average of Impact_on_Grades and Average of Predicted_Impact_on_Grades
by Daily_Usage_Hours
1.0
0.5
0.0
-0.5
1
2
3
4
5
Average of Impact_on_Grades
Average of Predicted_Impact_on_Grades

CHAPTER 8

ANNEXURE- PROGRESS SHEET

The progress sheet shows the weekly development and completion status of the project.

8.1 Weekly Progress Report

Week	Activity	Status
Week 1	Topic selection and problem definition	Completed
Week 2	Literature review and research study	Completed
Week 3	Synopsis preparation	Completed
Week 4	Feature identification and schema validation	Completed
Week 5	Data cleaning	Completed
Week 6	preprocessing pipline setup	Completed
Week 7	Feature matrix transformation	Completed
Week 8	TensorFlow Sequential Neural Network architecture construction	Completed
Week 9	Model training, loss optimization (MSE/Adam), and epoch tuning	Completed
Week 10	Testing and evaluation	Completed
Week 11	Power BI Dashboard construction (KPI Cards, Bar/Line Charts, Slicers)	Completed
Week 12	Complete Project Report documentation and chapter assembly	Completed

8.2 Work Distribution Among Team Members

Name	Responsibility
Mahima Chaudhari	Ingesting raw datasets, building Scikit-Learn preprocessing pipelines, designing and training TensorFlow Sequential Deep Learning models, calculating prediction errors, constructing Microsoft Power BI Desktop dashboards, and drafting technical project documentation.

8.3 Guide Interaction Record

Date	Discussion	Remarks
Week 1	Project topic approval	Approved
Week 3	Synopsis review	Suggested improvements
Week 6	Data processing review	Approved
Week 9	Model evaluation	Minor corrections
Week 12	Final submission	Approved